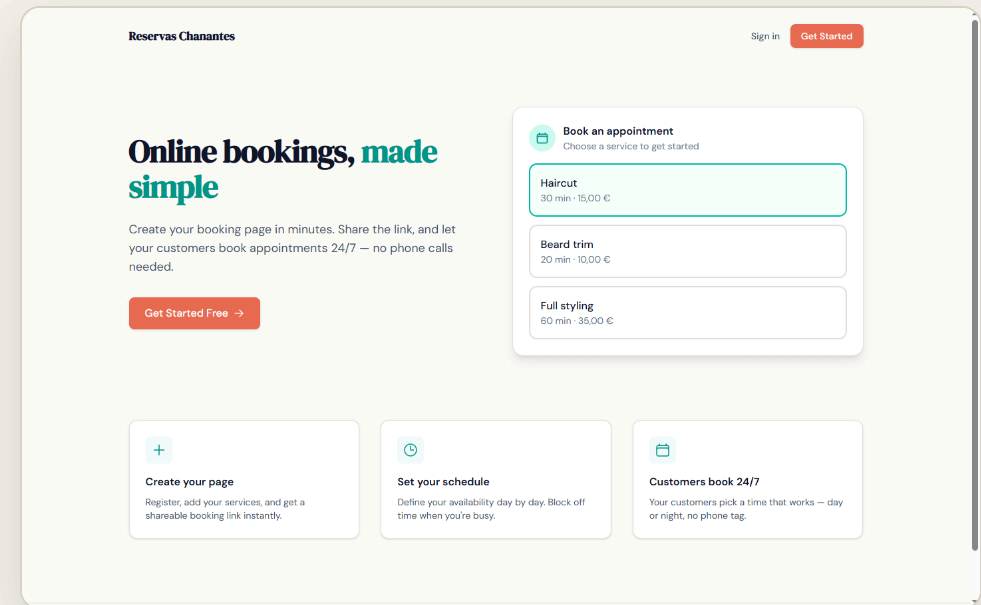


Reservas *Chanantes*

Plataforma SaaS *multi-tenant* de reserva de citas en línea para pequeños negocios de servicios.

Más que un producto: un caso de estudio sobre la viabilidad del desarrollo de software guiado por Inteligencia Artificial cuando se prioriza la máxima calidad de ingeniería.

► reservas-chanantes.vercel.app



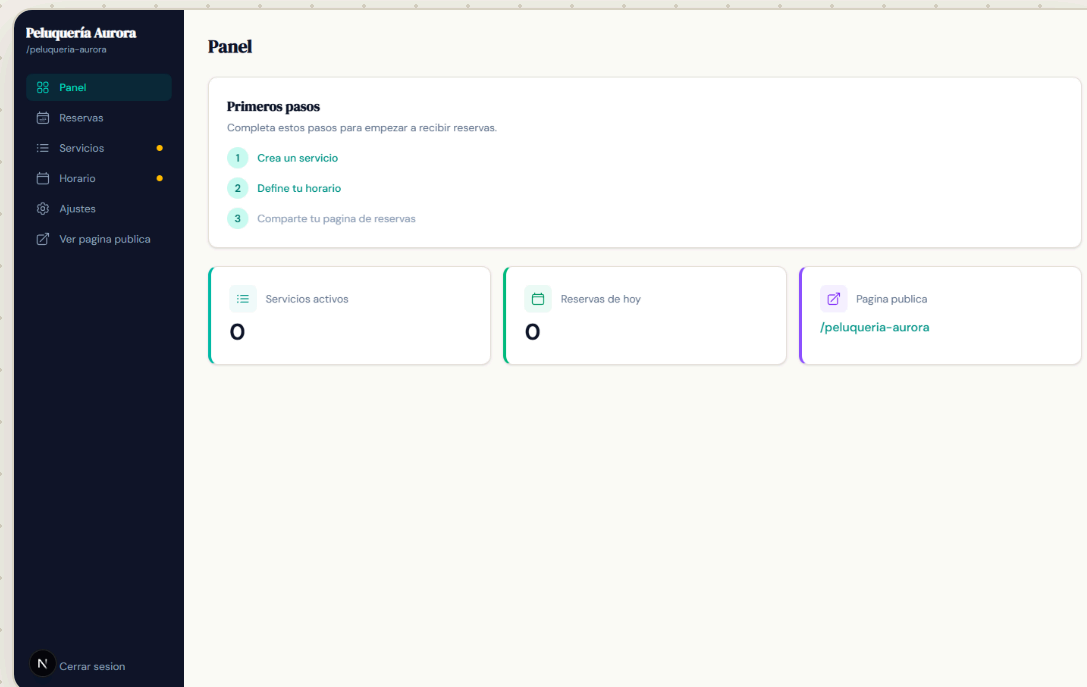
Las microempresas gestionan sus citas a mano.

- ◆ Peluquerías, fisioterapeutas, centros de estética... reservan por **teléfono y WhatsApp**.
- ◆ Sin un sistema que evite **solapamientos** ni calcule la disponibilidad de forma fiable.
- ◆ Sin **cobro integrado**: el pago se gestiona aparte, o no se gestiona.
- ◆ Las soluciones comerciales son **caras o demasiado genéricas** para este perfil.

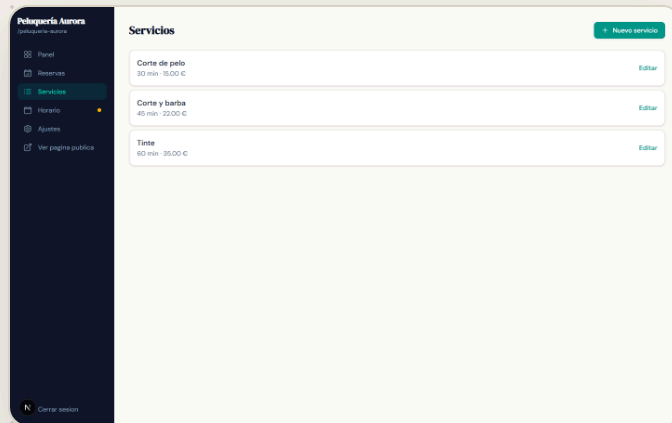
Cada negocio, su propio panel.

El negocio se registra como *tenant*, configura sus servicios y su horario, y publica una URL propia — **/peluqueria-aurora** — desde la que sus clientes reservan en tiempo real.

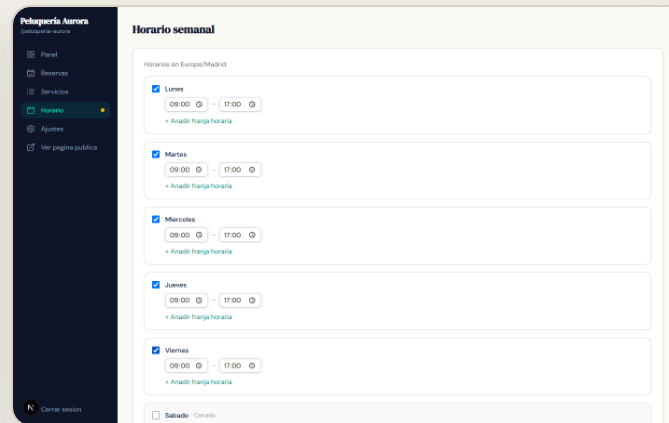
- ◆ **Panel** `/admin` — servicios, horario, reservas y cobros
- ◆ **Página pública** `/[slug]` — disponibilidad y reserva
- ◆ **Portal del cliente** `/my` — historial y autoservicio



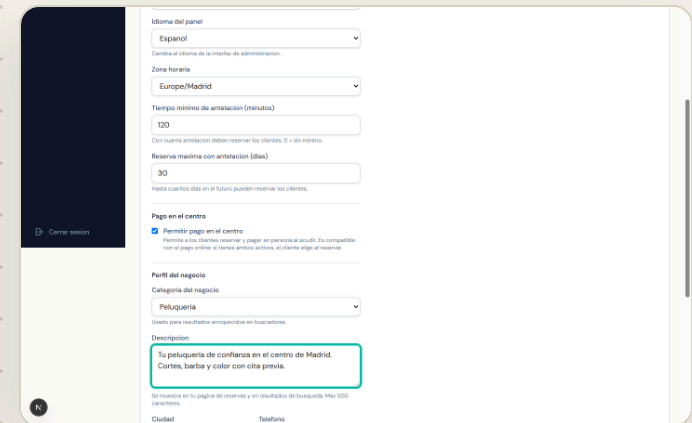
En marcha en minutos.



- Servicios y precios



- Horario semanal



- Pago: online o en el centro

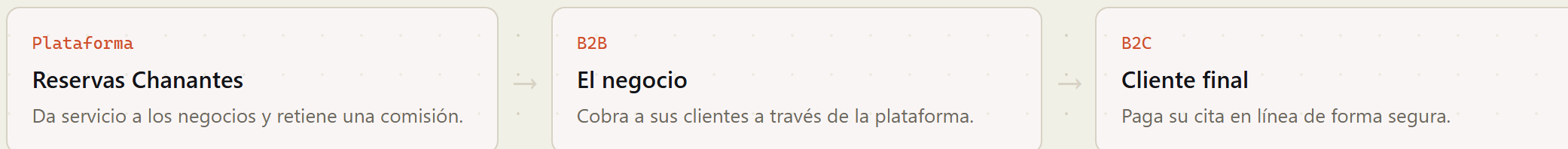
El propietario define su catálogo, su horario de apertura y cómo cobra — **pago online con Stripe, presencial en el centro, o ambos a la vez.**

El recorrido del cliente.



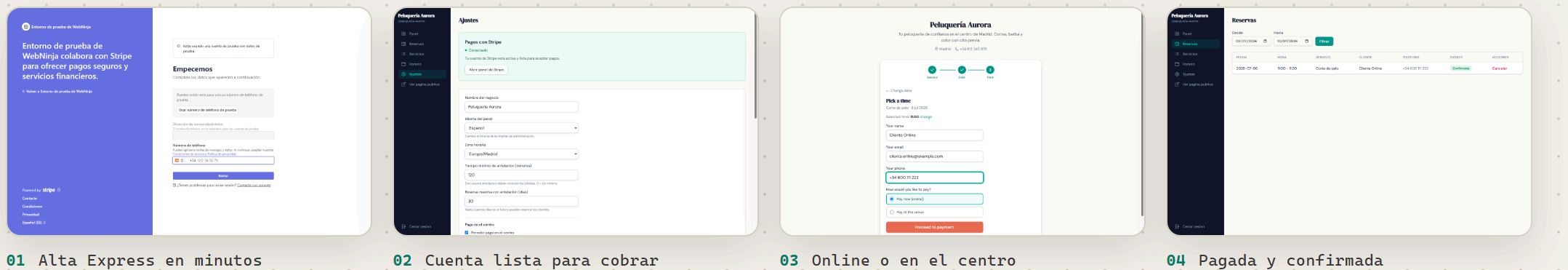
La disponibilidad se calcula al vuelo (horario – reservas), el hueco se **bloquea de inmediato** —sin dobles reservas—, el cliente paga **online o en el centro** y la cita aparece al instante **confirmada en el panel del negocio**.

B2B2C con pagos divididos.



- ◆ **Stripe Connect Express:** cada negocio conecta su cuenta con un *onboarding* ligero alojado por Stripe; el pago se confirma mediante *webhook* firmado.
- ◆ Comisión por plan diseñada en el dominio (`plan-limits`) — su activación se documenta como línea futura.

Cobrar online, sin fricción.



Con **Stripe Connect Express**, la plataforma crea la cuenta del negocio y lo lleva a un *onboarding* alojado por Stripe; el cobro es un **cargo directo** sobre su cuenta con **comisión** para la plataforma, y la reserva se confirma sola mediante *webhook* firmado.

El producto es el vehículo. La *tesis* es el método.

¿Es viable desarrollar software guiado por IA **sin renunciar a la máxima calidad de ingeniería**? Para responderlo, el sistema se construye bajo una disciplina explícita: **Clean Architecture · DDD · TDD · SOLID**, con trazabilidad documentada en planes, revisiones y registro de desviaciones.

Cuatro capas, una regla.

domain

Entidades, objetos de valor y servicios de dominio puros.

application

Casos de uso y puertos (interfaces de repositorio).

infrastructure

Adaptadores Supabase, Stripe, Resend.

presentation

Next.js: páginas, *Server Actions*, componentes.

La regla que lo sostiene todo: **el dominio no importa ninguna dependencia de *framework***. No es declarativo — es verificable, y es lo que habilita una pirámide de pruebas de base ancha.

Los problemas que no son triviales.

DISPONIBILIDAD

Horario de apertura menos reservas ocupadas, modelando el tiempo como *aritmética entera de minutos*.

CONCURRENCIA

Prevención de solapamientos en la capa autoritativa: restricción *EXCLUDE / btree_gist* de PostgreSQL → *SlotTakenError*.

ZONA HORARIA

Cálculo sensible a la zona horaria del negocio (*tenant-clock*), no del servidor.

IDEMPOTENCIA

Recordatorios sin duplicados ante reintentos del *cron*: patrón *claim-and-release*.

No se declara: se mide.

320

PRUEBAS AUTOMÁTICAS
EN 45 FICHEROS

96%

COBERTURA DE SENTENCIAS
(89 % RAMAS), UMBRAL EN CI

20

REQUISITOS FUNCIONALES
TRAZABLES AL CÓDIGO

11

MIGRACIONES SQL
VERSIONADAS

Pirámide de base ancha —dominio y aplicación concentran las pruebas—, habilitada por la arquitectura desacoplada. La cobertura con umbral, el pipeline **CI→CD encadenado** y las pruebas **E2E cross-browser** la convierten en calidad forzada por la herramienta.

Documentar lo que falta también es calidad.

LIMITACIONES RECONOCIDAS

- ◆ Monetización por plan **no activada**.
- ◆ Clean Architecture **parcial** en la rama de administración.
- ◆ Defecto conocido de **zona horaria** (getUTCDay).
- ◆ Sin pruebas de integración contra una **BD real** (se usan dobles).

Ya subsanadas respecto a versiones previas: la **RLS permisiva** (ahora acotada a propietario + titular) y la ausencia de **CI/CD y E2E**.

LÍNEAS FUTURAS PRIORIZADAS

- ◆ Modelado de amenazas formal, sobre el **paquete OWASP** ya aplicado.
- ◆ Pruebas de integración contra un **PostgreSQL efímero**.
- ◆ Activar monetización y sanear coherencia arquitectónica.
- ◆ Corregir zona horaria y mejorar **observabilidad** (APM).

Stack.

Next.js 16 · App Router

React 19

TypeScript estricto

Supabase · PostgreSQL + Auth + RLS

Stripe Connect

Resend · correo

Tailwind CSS 4

Vitest

Vercel · serverless

Pila moderna, de bajo coste operativo, elegida con criterios razonados frente a sus alternativas (Cap. 2 de la memoria).

El desarrollo asistido por IA, bajo *disciplina de ingeniería explícita*, es una vía viable para construir software no trivial y de calidad de producción.

Demo en vivo `reservas-chanantes.vercel.app`

Código `github.com/0gires/reservas-chanantes`

Memoria `docs/memoria/ · 7 capítulos + anexos`
